

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра вычислительной техники

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №7
ПО ДИСЦИПЛИНЕ «ПРОГРАММИРОВАНИЕ»

Факультет	ЗО	Преподаватель	Дубков Илья
Группа	ДТ-460а		Сергеевич
Студент	Пантюхин Артём Евгеньевич		
Вариант	9		

Новосибирск, 2026 г.

Цель работы

Ознакомиться с созданием шаблонов функций и классов. Изучить написание контейнерных шаблонных классов и их применение для построения различных структур данных.

Задание

Задание представляет собой типовую задачу по разработке шаблонов стандартных структур данных. Протестировать структуру данных. В качестве хранимых объектов использовать встроенные типы C++ (int и char).

Вариант 9:

Структура данных: двусвязный циклический список, содержащий элементы данных.

Операция: включение по заданному номеру.

Операция: поиск и возвращение элемента данных по заданному номеру

Исходный код

"list_node.hpp"

```
#ifndef S3L7_NODE_HPP
#define S3L7_NODE_HPP

namespace prog_s3 {
    template<typename T>
    struct Node {
        T value = T();
        Node* next = this;
        Node* previous = this;

        Node(): value(), next(this), previous(this) {};

        Node(T val, Node* next = nullptr, Node* prev = nullptr):
            value(val), next(next), previous(prev) {}

    };
}

#endif //S3L7_NODE_HPP
```

"my_list.hpp "

```
#ifndef S3L7_LIST_HPP
#define S3L7_LIST_HPP
```

```
#include "list_node.hpp"
```

```
#include <cstddef>
#include <initializer_list>
#include <limits>
#include <stdexcept>
```

//это старый лист из школы21, под задачу он подходит, пусть и чуть сложнее
требуемого

```
namespace prog_s3 {
template<typename T>
class list {
protected:
    using value_type = T;
    using reference = T &;
    using const_reference = const T &;
    using size_type = std::size_t;
    using pointer_type = T*;

    template <bool is_const>
    class BaseIterator {
    public:
        using value_ref = typename std::conditional<is_const,
const_reference, reference>::type;
        using node_ptr = typename std::conditional<is_const, const
Node<value_type>*, Node<value_type>*>::type;

        BaseIterator(): node_() {};
        BaseIterator(node_ptr node): node_(node) {}
        BaseIterator(const BaseIterator &other) = default;
        BaseIterator(BaseIterator &&other) = default;
        value_ref operator*() { return node_>value; }
        BaseIterator &operator++() {node_ = node_>next; return *this;}
        BaseIterator operator++(int) {node_ = node_>next; return
BaseIterator(node_>previous);};
        BaseIterator &operator--() {node_ = node_>previous; return *this;}
        BaseIterator operator--(int) {node_ = node_>previous; return
*BaseIterator(node_>next);};
        bool operator==(const BaseIterator &other) const {return this-
>node_ == other.node_;}
        bool operator!=(const BaseIterator &other) const {return !(this-
>node_ == other.node_);}

    protected:
        friend class list;
        node_ptr node_;
    };

    using ListIterator = BaseIterator<false>;
    using ListConstIterator = BaseIterator<true>;

    prog_s3::Node<value_type> header_;
    size_type size_;

    void insert_borrow(ListIterator pos, ListIterator other, size_type&
dest_size, size_type& src_size) {
        prog_s3::Node<value_type>* dest = pos.node_;
```

```

        prog_s3::Node<value_type>* src = other.node_;
        src->previous = dest->previous;
        src->next = dest;
        dest->previous = src;
        dest_size++;
        src_size--;
    }

    ListIterator get_offset_iterator(size_type offset) {
        ListIterator pos = begin();
        for (size_type i = 0; i < offset; i++) {
            pos++;
        }
        return pos;
    }

    reference get_offset_value(size_type offset) {
        return *get_offset_iterator(offset);
    }

    void set_offset_value(size_type offset, reference value) {
        *get_offset_iterator(offset) = value;
    }

    void swap_offset_values(size_type offset1, size_type offset2) {
        value_type value_holder = get_offset_value(offset1);
        set_offset_value(offset1, get_offset_value(offset2));
        set_offset_value(offset2, value_holder);
    }

    void remove_all_but_first(const_reference value) {
        bool first = true;
        while (auto i = begin() != end()) {
            if (*i == value) {
                if (first) first = !first;
                else erase(i++);
            }
        }
    }

    ListIterator qsort_divide(ListIterator part_low, ListIterator part_high) {
        ListIterator swap_point = part_low;
        value_type value_holder = value_type();
        while (part_low != part_high) {
            if (*part_low <= *part_high) {
                value_holder = *swap_point;
                *swap_point = *part_low;
                *part_low = value_holder;
                swap_point++;
            }
            part_low++;
        }

        value_holder = *swap_point;
        *swap_point = *part_high;
        *part_high = value_holder;

        return swap_point;
    }

    bool iterator_less(ListIterator source, ListIterator compare) {
        bool is_less = false;

```

```

        if (source != compare)
            while (source != end() && !is_less) {
                is_less = source == compare;
                source++;
            }
        return is_less;
    }

void qsort(ListIterator part_low, ListIterator part_high) {
    if (size_ < 2) return;
    if (iterator_less(part_low, part_high)) {
        ListIterator swap_point = qsort_divide(part_low, part_high);
        qsort(part_low, (--swap_point)++);
        qsort(++swap_point, part_high);
    }
}

public:
    using iterator = ListIterator;
    using const_iterator = ListConstIterator;

    list(): header_{}, size_(0) {};

    list(size_type n): list() {
        size_ = n;
        while (n) {
            push_back(value_type());
            n--;
        }
    }

    list(std::initializer_list<value_type> const &items): list() {
        for (auto item = items.begin(); item != items.end(); item++) {
            push_back(*item);
        }
    }

    list(const list &l) { //copy
        for (auto item = l.begin(); item != l.end(); item++) {
            push_back(*item);
        }
    }

    list(list &&l) { //move
        clear();
        header_ = std::move(l.header_);
        size_ = std::move(l.size_);
    }

    virtual ~list() { this->clear(); } //destructor

    list& operator=(list &&l) { //assignment move
        if (this != &l) {
            clear();
            header_ = std::move(l.header_);
            size_ = std::move(l.size_);
        }
        return *this;
    }

    //access methods
    const_reference front() { return begin().node_->value; }

```

```

const_reference back() { return end().node_->previous->value; }
const_reference at(size_type pos) {
    if (empty()) throw std::out_of_range("the list is empty");
    return get_offset_value(pos%size_);
}

//iterators
iterator begin() { return iterator(header_.next); }
iterator end() { return iterator(&header_); }

const_iterator cbegin() const { return const_iterator(header_.next); }
const_iterator cend() const { return const_iterator(&header_); }

//capacity
bool empty() {return size_ == 0; }
size_type size() { return size_; }
size_type max_size() { return std::numeric_limits<size_type>::max(); }

//mutators
void clear() {
    while (begin() != end())
        erase(begin());
}

iterator insert(iterator pos, const_reference value) {
    prog_s3::Node<value_type>* past = pos.node_->previous;
    prog_s3::Node<value_type>* next = pos.node_;
    prog_s3::Node<value_type>* alloc = new Node<value_type>(value, next,
past);

    past->next = alloc;
    next->previous = alloc;
    size_++;
    return iterator(alloc);
}

iterator insert_at(size_type pos, const_reference value) {
    size_type real_pos = size_ > 0 ? pos%(size_) : 0;
    //сдвинем в конец, чтобы было интуитивно понятно, что это новый элемент
    if (pos >= size_ && real_pos == 0) real_pos += size_;
    return insert(get_offset_iterator(real_pos), value);
}

void erase(iterator pos) { //may throw exception
    if (pos.node_ != nullptr && pos != end()) {
        pos.node_->next->previous = pos.node_->previous;
        pos.node_->previous->next = pos.node_->next;
        delete pos.node_;
        size_--;
    } else {
        throw std::out_of_range("erasing header_ || null");
    }
}

void push_back(const_reference value) {
    insert(end(), value);
}

void pop_back() { //remove last! (exceptional)
    erase(iterator(end().node_->previous));
}

void push_front(const_reference value) {

```

```

        insert(begin(), value);
    }

    void pop_front() { //remove first (exceptional)
        erase(begin());
    }

    void swap(list& other) {
        Node<value_type> temporary_header = header_;
        size_type temporary_size = size_;
        header_ = other.header_;
        size_ = other.size_;
        other.header_ = temporary_header;
        other.size_ = temporary_size;
    }

    void merge(list& other) { //we assume that both our lists are already
sorted (std behaviour)
        if (end() != other.end()) {
            while(other.begin() != other.end()) {
                auto dest_iter = begin();
                while (dest_iter != end() &&
std::less<value_type>(*(dest_iter++), *(other.begin())));
                insert_borrow(dest_iter, other.begin(), size_, other.size_);
            }
        }
    }

    void splice(const_iterator pos, list& other) { //transfer from other list
to this one, starting from pos.
        //if (end() != other.end()) //std assumes that those are different
lists anyway, behaviour is undefined
        while (other.begin() != other.end()) {
            insert_borrow(pos, other.begin(), size_, other.size_);
        }
    }

    void reverse() {
        iterator item = begin();
        while (item != end()) {
            prog_s3::Node<value_type>* tmp_node = item.node_->next;
            item.node_->next = item.node_->previous;
            item.node_->previous = item.node_->next;
            item--;
        }

        prog_s3::Node<value_type>* old_next = header_.next;
        header_.next = header_.previous;
        header_.previous = old_next;
    }

    void unique() { //deduplicate
        auto iter = begin();
        while (iter != end()) {
            remove_all_but_first(*iter);
        }
    }

    void sort() { qsort(begin(), --end()); }
}; //class list
} //namespace prog_s3

#endif //S3L7_LIST_HPP

```


"main.cpp"

```
#include "my_list.hpp"
```

```
#include <iostream>
```

```
/*
```

Этот контейнер - результат моего труда во время учёбы в школе²¹.
Он более комплексный, чем того требует задание, но было принято решение
не делать двойную работу, а использовать существующую реализацию.

В контейнер для данной ЛР добавлены методы insert_at() и at()

Что, в сущности, можно было бы сделать и унаследовав его...

```
*/
```

```
template <typename T>
```

```
void print_list(const prog_s3::list<T>& list) {
```

```
    for (auto i = list.cbegin(); i != list.cend(); i++) {  
        std::cout << *i << " ";
```

```
    }
```

```
    std::cout << std::endl;
```

```
}
```

```
void print_title(const char* title) {
```

```
    std::cout << "\033[32m|\\" << title << " |\\|\033[0m" << std::endl;
```

```
}
```

```
void print_closing(const char* title) {
```

```
    std::cout << "\033[32m|^|" << title << " |^|\033[0m" << std::endl;
```

```
}
```

```
void print_int() {
```

```
    print_title("Template container for int");
```

```
    prog_s3::list<int> list{1,2,3,4,5,6,7,8,9};
```

```
    std::cout << "Before insertion: " << std::endl;
```

```
    print_list(list);
```

```
    std::cout << "list.insert_at(552,777)" << std::endl;
```

```
    std::cout << "Expected: insertion at [3]" << std::endl;
```

```
    try {
```

```
        list.insert_at(552, 777);
```

```
    } catch(std::bad_alloc ex) {
```

```
        std::cerr << ex.what() << std::endl;
```

```
    }
```

```
    std::cout << "After insertion: " << std::endl;
```

```
    print_list(list);
```

```
    std::cout << std::endl << "list.at(3): " << list.at(3) << std::endl;
```

```
    print_closing("Template container for int");
```

```
}
```

```
void print_char() {
```

```
    print_title("Template container for char");
```

```
    prog_s3::list<char> list{'s','i','z','e'};
```

```
    std::cout << "Before insertion: " << std::endl;
```

```
    print_list(list);
```

```
    std::cout << "list.insert_at(8, 'd')" << std::endl;
```

```
    std::cout << "Expected: insertion at [4]" << std::endl;
```

```
    try {
```

```
        list.insert_at(8, 'd');
```

```
    } catch(std::bad_alloc ex) {
```

```
        std::cerr << ex.what() << std::endl;
```

```
    }
```

```

std::cout << "After insertion: " << std::endl;
print_list(list);
std::cout << "list.insert_at(0, 'r')" << std::endl;
std::cout << "list.insert_at(1, 'e')" << std::endl;
std::cout << "Expected: insertion of {r,e} at [0] and [1]" << std::endl;
try {
    list.insert_at(0, 'r');
    list.insert_at(1, 'e');
} catch(std::bad_alloc ex) {
    std::cerr << ex.what() << std::endl;
}
std::cout << "After insertion: " << std::endl;
print_list(list);

std::cout << std::endl << "list.at(34): " << list.at(34) << std::endl;

print_closing("Template container for char");
}

int main() {
    print_int();
    print_char();
    return 0;
}

```

Выводы

Реализован класс универсального шаблонного контейнера в виде двусвязного циклического списка. Несмотря на "циклическость", контейнер имеет чётко определённый корень, хранящий указатели на начало и конец, но не предназначенный для размещения в нём иных сущностей. Данная реализация позволяет существенно сократить объём работы, выполняемый как при модификации структуры списка, так и при работе со списком с использованием итераторов. Контейнер в процессе работы может вызвать два типа исключения – `std::bad_alloc` при ошибке выделения памяти и `std::out_of_range` при попытке получения элемента из пустого контейнера по индексу. В остальных случаях (например, при добавлении по итератору), результат работы подобен одноимённому контейнеру библиотеки STL.

В результате выполнения лабораторной работы были изучены процесс создания шаблонных функций и классов, логика и принцип работы контейнерных шаблонных классов, а так же способы их применения для построения различных структур данных.